Pandas

- **Pandas is a Python library used for data analysis and data manipulation.**

Why We Use Pandas

1.Easy to use

2.Works like Excel

3.Handles large data

4.Data cleaning

5. Data analysis

Pandas Types

1.Series

2.DataFrame

Series

Series is a one-dimensional labeled data structure in Pandas.

```
import pandas as pd

# create a Series
s = pd.Series(data, index)
```

- **data → list, array, or dictionary**
- **index → labels for each data value (optional)**

Properties of Pandas Series

1.values

```
# Returns all the data values in the Series (as a NumPy array)

import pandas as pd

s = pd.Series([10, 20, 30, 40])

print(s.values)
```

Output

```
[10 20 30 40]
```

2.index

Returns the index (labels) of the Series

```
import pandas as pd

s = pd.Series([100, 200, 300], index=["A", "B", "C"])

print(s.index)
```

Output

```
Index(['A', 'B', 'C'], dtype='object')
```

3.dtype

```
# Returns the data type of the elements in the Series

import pandas as pd

s = pd.Series([1, 2, 3, 4])

print(s.dtype)
```

Output

```
int64
```

4.shape

```
# Returns the shape (size) of the Series as a tuple

import pandas as pd

s = pd.Series([11, 22, 33, 44, 55])

print(s.shape)
```

Output

```
(5,)
```

5.ndim

```
# Returns the number of dimensions

import pandas as pd

s = pd.Series([5, 10, 15])

print(s.ndim)
```

Output

```
1
```

6.size

```
# Returns the total number of elements in the Series
```

```
import pandas as pd

s = pd.Series([2, 4, 6, 8, 10])

print(s.size)
```

Output

```
5
```

7.name

```
# Returns or sets the name of the Series

import pandas as pd

s = pd.Series([90, 80, 70], name="Marks")

print(s.name)
```

Output

```
Marks
```

8.empty

```
# Returns True if the Series is empty
```

```
import pandas as pd

s = pd.Series([])

print(s.empty)
```

Output

```
True
```

9.is_unique

```
# Checks if all values are unique

import pandas as pd

s = pd.Series([1, 2, 3, 4])

print(s.is_unique)
```

Output

```
True
```

10.hasnans

Checks whether the Series contains NaN values

```
import pandas as pd

s = pd.Series([10, 20, None, 40])
print(s.hasnans)
```

Output

```
True
```

11.nbytes

Shows memory used by Series values in bytes

```
import pandas as pd

s = pd.Series([1, 2, 3, 4])

print(s.nbytes)
```

Output

```
32
```

1 integer = 8 bytes

4 values × 8 = 32 bytes

Series-Methods

1.head()

```
# Returns the first n elements of the Series

# Default value of n = 5

import pandas as pd

s = pd.Series([10,20,30,40,50,60,70])
```

```
print(s.head())
```

Output

```
0    10
1    20
2    30
3    40
4    50
dtype: int64
```

```
     head(3)
# Returns first 3 elements

import pandas as pd

s =
pd.Series([10,20,30,40,50,60])

print(s.head(3))
```

Output

```
0    10
```

```
1     20
2     30
dtype: int64
```

2.tail()

```
# Returns the last n elements of the Series
# Default value n = 5

import pandas as pd

s = pd.Series([100,200,300,400,500,600,700])

print(s.tail())
```

Output

```
2    300
3    400
```

```
4    500
5    600
6    700
dtype: int64
```

```
            # tail(2)
# Returns last 2 elements

import pandas as pd

s = pd.Series([100,200,300,400,500])

print(s.tail(2))
```

Output

```
3    400
4    500
dtype: int64
```

3.sample()

```
# Returns random elements from the Series
import pandas as pd

s = pd.Series([10,20,30,40,50,60])

print(s.sample(2))
```

Output

```
4    50
1    20
dtype: int64
```

Output may change every time because sample() selects random values

4.unique()

```
# Returns unique values from the Series
```

```
import pandas as pd

s = pd.Series([10,20,10,30,20,40])

print(s.unique())
```

Output

```
[10 20 30 40]
```

5.nunique()

```
# Returns the count of unique values

import pandas as pd

s = pd.Series([1,2,2,3,4,4,5])

print(s.nunique())
```

Output

```
5
```

6.count()

```
# Returns number of non-null values
# NaN values are ignored

import pandas as pd
s = pd.Series([10,20,None,40,50])
print(s.count())
```

Output

```
4
```

7.value_counts()

```
# Returns frequency of each unique value

import pandas as pd

s = pd.Series([10,20,10,30,20,10])

print(s.value_counts())
```

Output

```
10    3
20    2
```

```
30    1
dtype: int64
```

8.Is_monotonic_increasing

```
# Returns True if values are in non-decreasing order
# (next value is greater than or equal to previous value)

import pandas as pd

s = pd.Series([10, 20, 20, 30, 40])

print(s.is_monotonic_increasing)
```

Output

```
True
```

is_monotonic_increasing Example with False

```
import pandas as pd

s = pd.Series([10, 50, 30, 40])

print(s.is_monotonic_increasing)
```

Output

```
False
```

9.is_monotonic_decreasing

```
# Returns True if values are in non-increasing order
# (next value is smaller than or equal to previous value)

import pandas as pd

s = pd.Series([50, 40, 40, 20, 10])

print(s.is_monotonic_decreasing)
```

Output

True

is_monotonic_decreasing Example with False

```
import pandas as pd
s = pd.Series([100, 80, 90, 70])
print(s.is_monotonic_decreasing)
```

Output

False

Sorting Methods

`sort_values()`

Sorts the Series values in ascending (default) or descending order.

Syntax

```
s.sort_values(ascending=True)
```

Example

```
import pandas as pd


s = pd.Series([40, 10, 30, 20], index=["d",
"a", "c", "b"])


print(s.sort_values())
```

Output

```
a    10

b    20

c    30

d    40

dtype: int64
```

Tracing

Original Series:

```
index:  d   a   c   b
value: 40  10  30  20
```

Sorted by value:

```
10 → index a
20 → index b
30 → index c
40 → index d
```

Descending Example

```
print(s.sort_values(ascending=False))
```

sort_index()

Sorts the index labels in alphabetical order (ascending by default).

Syntax

```
s.sort_index(ascending=True)
```

Example

```
print(s.sort_index())
```

Output

```
a    10
b    20
c    30
d    40
dtype: int64
```

Tracing

Original index order:

```
["d", "a", "c", "b"]
```

Sorted index order:

```
["a", "b", "c", "d"]
```

Descending Example

```
print(s.sort_index(ascending=False))
```

duplicated()

Returns True/False for each element telling whether the value is a duplicate.

```
Series.duplicated(keep='first')
```

- `"first"` → **mark duplicates except first occurrence**
- `"last"` → **mark duplicates except last occurrence**
- `False` → **mark all duplicates as True**

Example

```
import pandas as pd


s = pd.Series([10, 20, 10, 30, 20, 40])


print(s.duplicated())
0    False   # first 10 → not duplicate
1    False   # first 20 → not duplicate
2     True   # second 10 → duplicate
3    False
4     True   # second 20 → duplicate
5    False
```

drop_duplicates()

Removes duplicate values from the Series and returns only unique values.

```
Series.drop_duplicates(keep='first')
```

Example

```
s = pd.Series([10, 20, 10, 30, 20, 40])
print(s.drop_duplicates())
```

output

```
0    10
1    20
3    30
5    40
```

where()

Keeps values where condition is True, otherwise replaced with another value (default: NaN).

```
Series.where(condition, other=None)
```

Example

```
s = pd.Series([10, 20, 30, 40])
```

```
print(s.where(s > 20))
```

Output

```
0     NaN
1     NaN
2    30.0
3    40.0
```

→ Only values > 20 are kept; others become NaN.

```
s.where(s > 20, 0)
```

Output:

```
0     0
1     0
2    30
3    40
```

---

mask()

Opposite of `where()`
Replaces values where condition is True, keeps values otherwise.

```
Series.mask(condition, other=None)
```

Example

```
s = pd.Series([10, 20, 30, 40])


print(s.mask(s > 20))
```

Output

```
0    10
1    20
2     NaN
3     NaN
```

```
s.mask(s > 20, 0)
```

Output:

```
0    10
1    20
2     0
3     0
```

Basic Arithmetic Operators

Operation	Symbol	Method	Meaning
Addition	`+`	`add()`	s1 + s2
Subtraction	`-`	`sub()`	s1 - s2
Multiplication	`*`	`mul()`	s1 * s2
Division	`/`	`div()`	s1 / s2
Floor Division	`//`	`floordiv()`	s1 // s2
Modulus	`%`	`mod()`	s1 % s2
Power	`**`	`pow()`	s1 ** s2

Example

```
import pandas as pd

s1 = pd.Series([10, 20, 30], index=['a','b','c'])
s2 = pd.Series([1, 2, 3], index=['a','b','c'])

print("Addition (+):")
print(s1 + s2)
```

```
print("\nSubtraction (-):")
print(s1 - s2)

print("\nMultiplication (*):")
print(s1 * s2)

print("\nDivision (/):")
print(s1 / s2)

print("\nFloor Division (//):")
print(s1 // s2)

print("\nModulus (%):")
print(s1 % s2)

print("\nPower (**):")
print(s1 ** s2)
```

Extra Informations

nlargest()

Returns the top n largest values from the Series.

Syntax

```
s.nlargest(n)
```

Example

```
import pandas as pd


s = pd.Series([50, 10, 80, 30,
90, 20])
```

```
print(s.nlargest(3))
```

Output

```
4    90
2    80
0    50
dtype: int64
```

nsmallest()

Returns the bottom n smallest values.

Syntax

```
s.nsmallest(n)
```

Example

```
s = pd.Series([50, 10, 80, 30,
90, 20])


print(s.nsmallest(3))
```

Output

```
1    10

5    20

3    30

dtype: int64
```

take()

Selects elements by integer positions (not by labels).

Syntax

```
s.take([index_positions])
```

Example

```
s = pd.Series([10, 20, 30, 40, 50])


print(s.take([0, 3, 4]))
```

Output

```
0    10

3    40

4    50

dtype: int64
```

get()

Returns value at index label, returns default value if label does not exist.

Syntax

```
s.get(label, default)
```

Example

```
s = pd.Series([100, 200, 300], index=["a",
"b", "c"])


print(s.get("b"))

print(s.get("x", "Not Found"))
```

Output

```
200

Not Found
```

loc

Label-based indexing.
Use .loc when you want to select using index names.

Syntax

```
s.loc[label]

s.loc[start:end]
```

Example

```
s = pd.Series([10, 20, 30, 40], index=["a", "b", "c", "d"])


print(s.loc["b"])

print(s.loc["b":"d"])
```

Output

```
20

b    20

c    30
```

```
d    40
dtype: int64
```

iloc

Position-based indexing.
Use .iloc when you want to select using integer positions.

Syntax

```
s.iloc[position]
s.iloc[start:end]
```

Example

```
s = pd.Series([10, 20, 30, 40])

print(s.iloc[1])     # element at position 1
```

```
print(s.iloc[1:3])   # slice
```

max()

Definition

Returns the maximum value.

Example

```
x = pd.Series([10,50,30])

print(x.max())
```

Output

50

min()

Definition

Returns the minimum value.

Example

```
x = pd.Series([10,50,30])

print(x.min())
```

sum()

Definition

Returns the total sum of all values.

Example

```
x = pd.Series([10,20,30])

print(x.sum())
```

Output

60

mean()

Definition

Returns average value.

```
x = pd.Series([10,20,30])

print(x.mean())
```

Output

20.0

median()

Definition

Returns middle value after sorting.

Example

```
x = pd.Series([50,10,30])

print(x.median())
```

Output

30.0

mode()

Definition

Returns the most repeated value.

Example

```
x = pd.Series([10,20,20,30])

print(x.mode())
```

idxmax()

Definition

Returns index position of maximum value.

Example

```
x = pd.Series([10,80,30])

print(x.idxmax())
```

Output

```
1
```

idxmin()

Definition

Returns index position of minimum value.

Example

```
x = pd.Series([10,80,30])

print(x.idxmin())
```

Output

```
0
```

astype()

Definition

Converts datatype of Series.

Example

```
x = pd.Series([1,2,3])

print(x.astype(float))
```

drop()

Definition

Removes selected index value.

Example

```
x = pd.Series([10,20,30],index=["a","b","c"])

print(x.drop("b"))
```

pop()

Definition

Removes and returns value from given index.

Example

```
x = pd.Series([10,20,30])

print(x.pop(1))
print(x)
```

fillna()

Definition

Replaces missing values.

Example

```
x = pd.Series([10,None,30])

print(x.fillna(0))
```

replace()

Definition

Replaces old value with new value.

Example

```
x = pd.Series([10,20,30])

print(x.replace(20,99))
```

between()

Definition

Checks values between given range.

Example

```
x = pd.Series([10,20,30,40])

print(x.between(15,35))
```

dropna()

Definition

Removes missing values.

Example

```
x = pd.Series([10,None,30])

print(x.dropna())
```

del keyword

Definition

Deletes Series variable completely.

Example

```
x = pd.Series([10,20,30])

del x
```

rename()

Definition

Changes Series name or index labels.

Example

```
x = pd.Series([10,20,30])

x = x.rename("Numbers")

print(x)
```

duplicated()

Definition

Returns True for duplicate values.

Example

```
x = pd.Series([10,20,20,30])

print(x.duplicated())
```

drop_duplicates()

Definition

Removes duplicate values.

Example

```
x = pd.Series([10,20,20,30])

print(x.drop_duplicates())
```

Indexing

Definition

Accessing a single value using an index.

Example

```
x = pd.Series([10,20,30])

print(x[1])
```

Slicing

Definition

Accessing multiple values using range.

Example

```
x = pd.Series([10,20,30,40,50])

print(x[1:4])
```

isnull()

Definition

Checks missing values.

Example

```
x = pd.Series([10,None,30])

print(x.isnull())
```

notnull()

Definition

Checks non-missing values.

Example

```
x = pd.Series([10,None,30])

print(x.notnull())
```

26. nlargest()

Definition

Returns largest values.

Example

```
x = pd.Series([10,50,30,80])

print(x.nlargest(2))
```

27. nsmallest()

Definition

Returns smallest values.

Example

```
x = pd.Series([10,50,30,80])

print(x.nsmallest(2))
```

28. take()

Definition

Returns values using position indexes.

Example

```
x = pd.Series([10,20,30,40])

print(x.take([0,2]))
```

29. get()

Definition

Gets value safely from index.

Example

```
x = pd.Series([10,20,30])

print(x.get(1))
```

30. where()

Definition

Keeps values if condition True otherwise replaces.

Example

```
x = pd.Series([10,20,30,40])

print(x.where(x>20))
```

31. mask()

Definition

Replaces values where condition is True.

Example

```
x = pd.Series([10,20,30,40])

print(x.mask(x>20))
```

32. tolist()

Definition

Converts Series into Python list.

Example

```
x = pd.Series([10,20,30])

print(x.tolist())
```

33. info()

Definition

Gives complete information about Series.

Example

```
x = pd.Series([10,20,30])

print(x.info())
```

34. describe()

Definition

Returns statistical summary.

Example

```
x = pd.Series([10,20,30,40])

print(x.describe())
```

35. cumsum()

Definition

Returns cumulative sum.

Example

```
x = pd.Series([10,20,30])

print(x.cumsum())
```

Output

```
0    10
1    30
2    60
dtype: int64
```

36. cumprod()

Definition

Returns cumulative product.

Example

```
x = pd.Series([2,3,4])

print(x.cumprod())
```

Output

```
0     2
1     6
2    24
dtype: int64
```

#Most Important Series Methods List

head()
tail()
sample()
describe()
info()
sum()
mean()
median()
mode()
max()
min()
count()
unique()
nunique()
value_counts()
sort_values()
sort_index()
isnull()

notnull()
dropna()
fillna()
astype()
replace()
drop()
pop()
where()
mask()
tolist()
to_numpy()
to_dict()
duplicated()
between()
nlargest()
nsmallest()
loc[]
lloc[]

'

#Most Important Series Properties

values

index
dtype
shape
size`
ndim
empty
hasnans
T
axes
nbytes
name
is_unique
is_monotonic_increasing

is_monotonic_decreasing

DataFrame

DataFrame is a two-dimensional labeled data structure in Pandas that stores data in rows and columns

OR

DataFrame is a table-like data structure in Pandas that stores data in rows and columns.

Creating DataFrame

1. Using Dictionary

```
import pandas as pd

data = {
    "Name": ["Ram","Ravi","Sita"],
    "Age": [21,22,20],
    "Marks": [85,90,88]
}

df = pd.DataFrame(data)

print(df)
```

2. From List

```
data = [["Ram",21,85],
        ["Ravi",22,90],
        ["Sita",20,88]]
```

```
df = pd.DataFrame(data, columns=["Name","Age","Marks"])
print(df)
```

Example

```
import pandas as pd

data={
   "Name":["Ram","Ravi","Sita"],
   "Age":[21,22,20],
   "Marks":[85,90,88]
}

df=pd.DataFrame(data)
```

print(df)

DataFrame-Properties

Refer Above example

1. shape

Definition

Returns the number of rows and columns in the DataFrame.

Syntax

```
df.shape
```

Example

```
print(df.shape)
```

Output

```
(3, 3)
```

2. size

Definition

Returns total number of elements in DataFrame.

Example

```
print(df.size)
```

Output

```
9
```

3. index

Definition

Returns row labels/index numbers of DataFrame.

Example

```
print(df.index)
```

Output

```
RangeIndex(start=0, stop=3,
step=1)
```

4. columns

Definition

Returns column names of DataFrame.

Example

```
print(df.columns)
```

Output

```
Index(['Name', 'Age', 'Marks'],
dtype='object')
```

5. dtypes

Definition

Returns data type of each column.

Example

```
print(df.dtypes)
```

Output

```
Name     object
Age       int64
Marks     int64
dtype: object
```

6. values

Definition

Returns all DataFrame data in NumPy array format.

Example

```
print(df.values)
```

Output

```
[['Ram' 21 85]
```

['Ravi' 22 90]
['Sita' 20 88]]

7. ndim

Definition

Returns dimension of DataFrame.

Example

print(df.ndim)

Output

2

8. empty

Definition

Checks whether DataFrame is empty or not.

Example

print(df.empty)

Output

False

9. axes

Definition

Returns row indexes and column names together.

Example

print(df.axes)

Output

[RangeIndex(start=0, stop=3, step=1), Index(['Name', 'Age', 'Marks'], dtype='object')]

10. T (Transpose)

Definition

Converts rows into columns and columns into rows.

Example

print(df.T)

Output

```
         0      1      2
Name    Ram   Ravi  Sita
Age      21     22    20
Marks    85     90    88
```

Example 02

```
import pandas as pd

data={

"Name":["Ram","Ravi","Sita","Kiran","Asha"],
    "Age":[21,22,20,23,24],
    "Marks":[85,90,88,76,95]
}

df=pd.DataFrame(data)
```

```
print(df)

    Name  Age  Marks
0    Ram   21     85
1   Ravi   22     90
2   Sita   20     88
3  Kiran   23     76
4   Asha   24     95
```

1. head()

Definition

Returns the first rows of the DataFrame.

Default:
Returns the first 5 rows.

Syntax

df.head()

Example

print(df.head())

Output

```
   Name  Age  Marks
0    Ram   21     85
1   Ravi   22     90
2   Sita   20     88
3  Kiran   23     76
4   Asha   24     95
```

Selected Rows

print(df.head(2))

Output

```
  Name  Age  Marks
0   Ram   21     85
1  Ravi   22     90
```

2. tail()

Definition

Returns the last rows of the DataFrame.

Default:
Returns last 5 rows.

Syntax

```
df.tail()
```

Example

```
print(df.tail(2))
```

Output

```
   Name  Age  Marks
3  Kiran   23     76
4   Asha   24     95
```

3. sample()

Definition

Returns random rows from the DataFrame.

Syntax

df.sample(n)

Example

print(df.sample(2))

Possible Output

```
   Name  Age  Marks
1   Ravi   22     90
4   Asha   24     95
```

Note:

Every execution may give different random rows.

4. info()

Definition

Returns complete summary information about DataFrame.

It shows:

- **Number of rows**
- **Column names**
- **Non-null values**
- **Data types**
- **Memory usage**

Syntax

df.info()

Example

df.info()

Output

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 3 columns):
Name     5 non-null object
Age      5 non-null int64
Marks    5 non-null int64
dtypes: int64(2), object(1)
```

5. describe()

Definition

Returns statistical summary of numerical columns.

It shows:

- **count**
- **mean**
- **std**
- **min**
- **max**
- **percentile values**

Syntax

df.describe()

Example

print(df.describe())

Output

	Age	Marks
count	5.000000	5.000000
mean	22.000000	86.800000
std	1.581139	7.224957
min	20.000000	76.000000
25%	21.000000	85.000000
50%	22.000000	88.000000
75%	23.000000	90.000000
max	24.000000	95.000000

Understanding loc[ ]

(Stop Index Included)

Definition

`loc[]` is used to access data using:

- Row labels/index names
- Column names

In `loc[]`, stop index is included.

1. Single Row Selection

Syntax

df.loc[row_number]

Example

print(df.loc[1])

Output

Name Ravi

Age 22

Marks 90

2. Single Row with Selected Columns

Syntax

df.loc[row_number,["column_name"]]

Example

```
print(df.loc[1,["Name","Marks"]])
```

Output

```
Name     Ravi
Marks      90
```

3. Row Range Selection

Syntax

```
df.loc[start:stop]
```

Example

```
print(df.loc[1:3])
```

Output

```
   Name  Age  Marks
1   Ravi   22     90
2   Sita   20     88
3  Kiran   23     76
```

4. Row Range with Single Column

Syntax

```
df.loc[start:stop,"column_name"]
```

Example

```
print(df.loc[1:3,"Name"])
```

Output

```
1     Ravi
2     Sita
3    Kiran
```

5. Row Range with Multiple Columns

Syntax

```
df.loc[start:stop,["column1","column2"]]
```

Example

```
print(df.loc[1:3,["Name","Marks"]])
```

Output

```
   Name  Marks
1   Ravi     90
2   Sita     88
3  Kiran     76
```

6. Column Range Selection

Syntax

```
df.loc[start:stop,"column1":"column_last"]
```

Example

```
print(df.loc[1:3,"Name":"Marks"])
```

Output

```
   Name  Age  Marks
1   Ravi   22     90
2   Sita   20     88
3  Kiran   23     76
```

Understanding iloc[ ]

(Stop Index Excluded)

Definition

`iloc[]` is used to access data using:

- Row numbers
- Column numbers

In `iloc[]`, the stop index is excluded.

1. Single Row and Column

Syntax

```
df.iloc[row_number,column_number]
```

Example

```
print(df.iloc[1,2])
```

Output

```
90
```

2. Row Range and Column Range

Syntax

```
df.iloc[row_start:row_stop,col_start:col_stop]
```

Example

```
print(df.iloc[1:4,0:2])
```

Output

```
   Name  Age
1   Ravi   22
2   Sita   20
3  Kiran   23
```

3. Row Range with Single Column

Syntax

```
df.iloc[start:stop,column_number]
```

Example

```
print(df.iloc[1:4,0])
```

Output

1 Ravi

2 Sita

3 Kiran

4. Multiple Selected Rows

Syntax

```
df.iloc[[row1,row2]]
```

Example

```
print(df.iloc[[0,2,4]])
```

Output

```
  Name  Age  Marks
0  Ram   21     85
2 Sita   20     88
4 Asha   24     95
```